

Chess Game in C++ Using Object-Oriented Programming

1. Introduction

This project is a console-based Chess Game developed in C++ using Object-Oriented Programming (OOP) principles. The aim of this project is to simulate a two-player chess game where users can play by following standard chess rules.

The system is designed to validate moves, enforce rules such as check, checkmate, stalemate, castling, and pawn promotion, and provide an interactive gameplay experience using a text-based interface.

2. Objectives

- To develop a fully functional chess game in C++
 - To implement core OOP concepts
 - To simulate real chess rules in a console environment
 - To ensure memory management using dynamic allocation
-

3. OOP Concepts Used

Encapsulation

Encapsulation is implemented through the use of classes such as piece, pawn, rook, knight, bishop, queen, king, and board.

- Data members like color and symbol are protected
 - Access is controlled using public member functions like `getcolor()` and `getsymbol()`
-

Inheritance

Inheritance is used by creating a base class piece and deriving all chess pieces from it:

- pawn
- rook
- knight

- bishop
- queen
- king

Each derived class inherits common attributes and overrides behavior.

Polymorphism

Polymorphism is implemented using virtual functions:

- The base class defines:
- virtual bool isvalidmove(...)
- Each derived class overrides this function according to its movement rules

This allows dynamic behavior depending on the object type.

Composition

The board class demonstrates composition:

- It contains a 2D array:
 - piece* matrix[8][8];
 - It manages all chess pieces and their positions
-

4. Project Design

Base Class: piece

- Stores color and symbol
 - Defines virtual function isvalidmove()
-

Derived Classes

Each piece implements its own movement logic:

- **Pawn** → forward movement, double move, diagonal capture, promotion

- **Rook** → horizontal and vertical movement
 - **Knight** → L-shaped movement
 - **Bishop** → diagonal movement
 - **Queen** → combination of rook and bishop
 - **King** → one-step movement + castling
-

Board Class

The board class is the core controller:

- Initializes the board
 - Displays the board
 - Validates moves
 - Detects:
 - Check
 - Checkmate
 - Stalemate
 - Handles special moves:
 - Castling
 - Pawn Promotion
-

5. Features Implemented

- Full 8×8 chess board display
- Two-player turn-based system
- Move validation for all pieces
- Check detection
- Checkmate detection
- Stalemate detection

- Castling implementation
 - Pawn promotion to Queen
 - Memory management using dynamic allocation and destructor
-

6. How the Game Works

1. The board is initialized with all pieces
 2. Players take turns (White starts first)
 3. Input format:
 4. Current position: e2
 5. Moving position: e4
 6. The system:
 - Validates move
 - Updates board
 - Checks game conditions
 7. Game ends when:
 - Checkmate occurs
 - Stalemate occurs
-

7. Challenges Faced

- Implementing correct movement logic for each piece
 - Managing dynamic memory safely
 - Handling complex rules like castling and check detection
 - Preventing illegal moves that expose the king
-

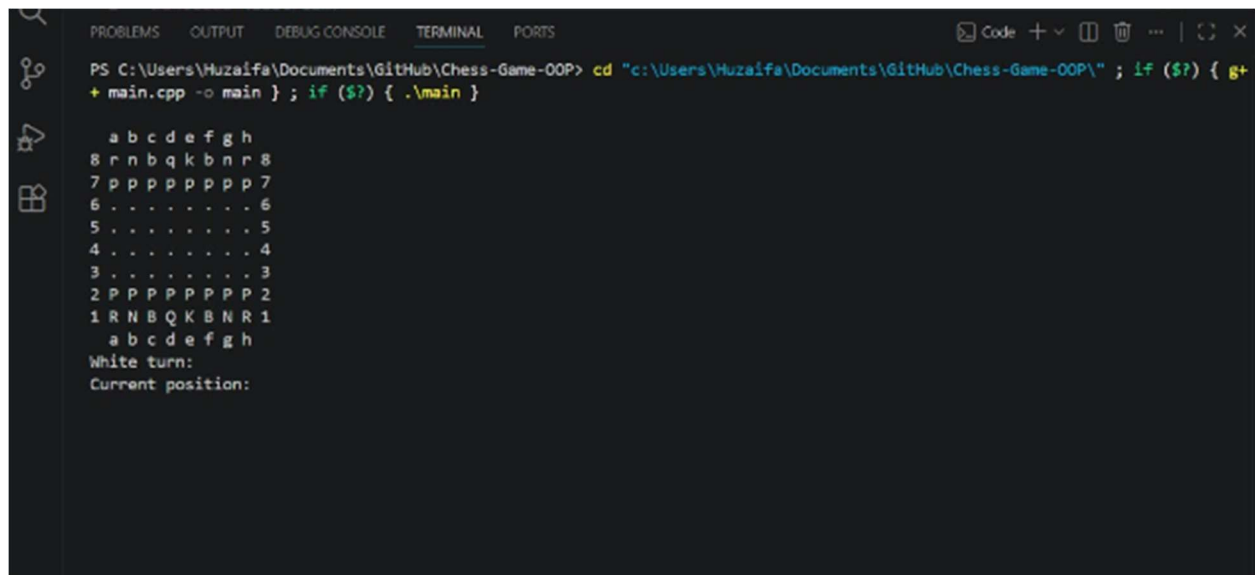
8. Conclusion

This project successfully demonstrates a complete chess game using Object-Oriented Programming in C++. It highlights the use of inheritance, polymorphism, and composition in solving a real-world problem. The implementation ensures correct gameplay mechanics and efficient memory handling.

9. Future Improvements

- Add graphical user interface (GUI)
 - Implement AI opponent
 - Add undo/redo functionality
 - Improve user input handling
-

10. Screen Shots



```
PS C:\Users\HuzaiFa\Documents\GitHub\Chess-Game-OOP> cd "c:\Users\HuzaiFa\Documents\GitHub\Chess-Game-OOP\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
```

```
  a b c d e f g h
8 r n b q k b n r 8
7 p p p p p p p 7
6 . . . . . 6
5 . . . . . 5
4 . . . . . 4
3 . . . . . 3
2 P P P P P P P 2
1 R N B Q K B N R 1
  a b c d e f g h
White turn:
Current position:
```

```
1 #include <iostream>
2
3 Moving position: d
4
5   a b c d e f g h
6 8 r n b q k b n r 8
7 7 p p p p p p p 7
8 6 . . . . . 6
9 5 . . . . . 5
10 4 . . . . . 4
11 3 . . P . . . . 3
12 2 P P P . P P P 2
13 1 R N B Q K B N R 1
14   a b c d e f g h
15 Black turn:
16 Current position: c
17 7
18 Moving position: c
19 6
20
21   a b c d e f g h
22 8 r n b q k b n r 8
23 7 p p . p p p p 7
24 6 . . p . . . . 6
25 5 . . . . . 5
26 4 . . . . . 4
27 3 . . P . . . . 3
28 2 P P P . P P P 2
29 1 R N B Q K B N R 1
30   a b c d e f g h
31 White turn:
32 Current position: 
```

```
1 #include <iostream>
2
3 6
4 Moving position: c
5 5
6
7   a b c d e f g h
8 8 r n b q k b n r 8
9 7 p p . p p p p 7
10 6 . . . . . 6
11 5 . . p . . . . 5
12 4 . . P . . . . 4
13 3 . . . . . 3
14 2 P P P . P P P 2
15 1 R N B Q K B N R 1
16   a b c d e f g h
17 White turn:
18 Current position: d
19 4
20 Moving position: c
21 5
22
23   a b c d e f g h
24 8 r n b q k b n r 8
25 7 p p . p p p p 7
26 6 . . . . . 6
27 5 . . P . . . . 5
28 4 . . . . . 4
29 3 . . . . . 3
30 2 P P P . P P P 2
31 1 R N B Q K B N R 1
32   a b c d e f g h
33 Black turn:
34 Current position: 
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Huzaiifa\Documents\GitHub\Chess-Game-OOP> cd "c:\Users\Huzaiifa\Documents\GitHub\Chess-Game-OOP\" ; if ($?) { g+
+ main.cpp -o main } ; if ($?) { .\main }

  a b c d e f g h
8 r n b q k b n r 8
7 p p p p p p p 7
6 . . . . . 6
5 . . . . . 5
4 . . . . . 4
3 . . . . . 3
2 P P P P P P P 2
1 R N B Q K B N R 1
  a b c d e f g h
White turn:
Current position: d
2
Moving position: d
3

  a b c d e f g h
8 r n b q k b n r 8
7 p p p p p p p 7
6 . . . . . 6
5 . . . . . 5
4 . . . . . 4
3 . . . P . . . 3
2 P P P . P P P 2
1 R N B Q K B N R 1
  a b c d e f g h
Black turn:
Current position: █
```

11. Members

- Member 1: Muhammad Huzaifa Siddiqi 25F-3071
- Member 2: Fasih Siddeeq 25F-3074
- Member 3: Abdullah Shahzad 25F-3077